

Práctica 4

SonarLint

Nombre y Apellidos:

Objetivos

- Aprender a usar SonarLint para mejorar la calidad del código

Materiales y Herramientas Requeridas

- Ordenador con JDK (Java Development Kit) instalado (versión 17 o superior).
- Maven
- Editor de texto o IDE (Entorno de Desarrollo Integrado) de preferencia (IntelliJ Idea, Eclipse)
- Plugin de SonarLint

Ejercicios

Ejercicio 1: Instalación de SonarLint en el IDE de tu elección

1. Seguir las instrucciones de instalación

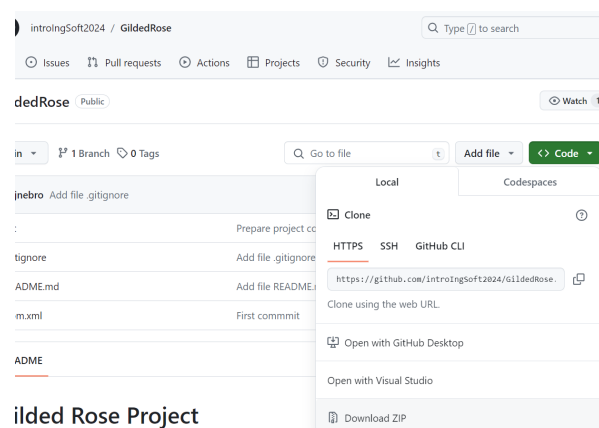
Eclipse: <https://docs.sonarsource.com/sonarlint/eclipse/getting-started/installation/>

IntelliJ: <https://docs.sonarsource.com/sonarlint/intellij/getting-started/installation/>

Ejercicio 2: Descargar el proyecto GildedRose del repositorio en GitHub

1. Entrar en GitHub y descargar el proyecto:

<https://github.com/introIngSoft2024/GildedRose>

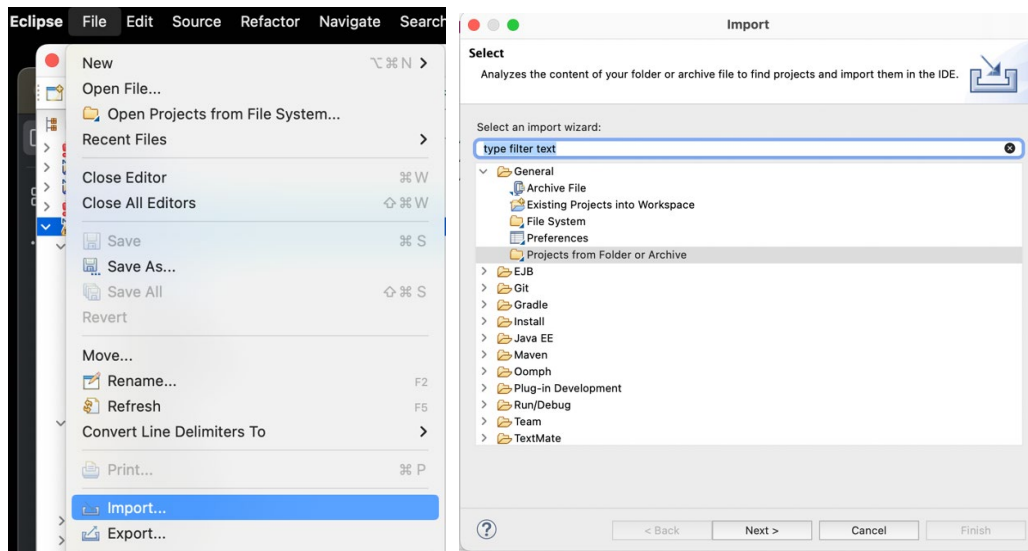


2. **Descomprimir el fichero.** Descomprimir el fichero en local y comprobar que el contenido del proyecto es el mismo que el que está en GitHub.

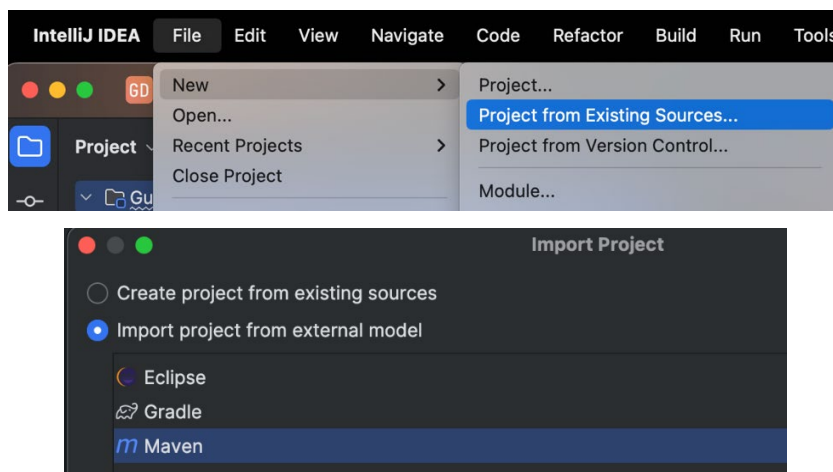
Ejercicio 3: Revisar el proyecto en el IDE

1. Importar el proyecto.

Eclipse: Seleccionar File -> Import -> Projects from folder or archive y seguir las instrucciones.



IntelliJ: Seleccionar File -> New -> Project from Existing Sources, seleccionar la carpeta con el proyecto descomprimido y después seleccionar Import Project From External Model y la opción de Maven

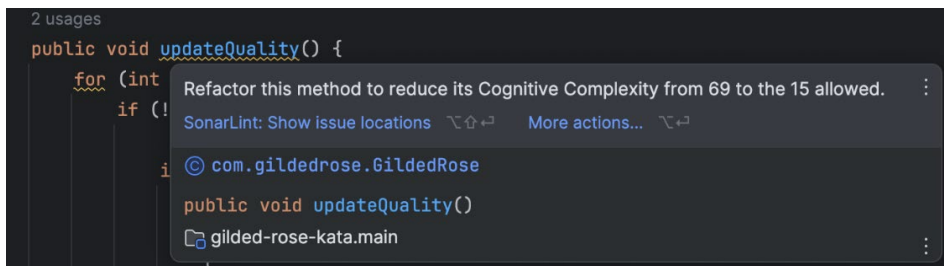
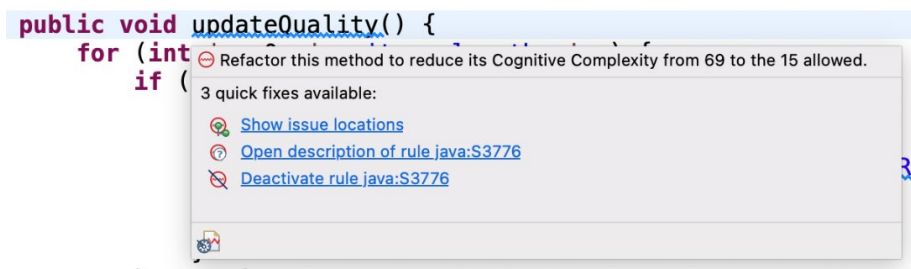


2. Dentro de la carpeta *main*, con el código del proyecto, hay una carpeta *original* y otra carpeta *refactored*. Lo que toquemos será lo de la carpeta *refactored*.
3. **Leer documentación.** Accede al fichero README.md y selecciona el enlace para leer los requisitos en el idioma que desees. Tómate tu tiempo en entenderlos.
4. **Entender el código.** Abre la clase GildedRose.java e intenta entender el código.

Ejercicio 4: Mejorar los identificadores y las constantes

1. **Leer sugerencias de SonarLint.** En el código, las sugerencias de SonarLint aparecen subrayadas en azul o con un fondo de otro color. Sitúa el cursor encima de ellas para que aparezca la sugerencia de SonarLint.

```
public void updateQuality() {
    for (int i = 0; i < items.length; i++) {
        if (!items[i].name.equals("Aged Brie")
            && !items[i].name.equals("Backstage passes to a TAFKAL80ETC concert")) {
            if (items[i].quality > 0) {
                if (!items[i].name.equals("Sulfuras, Hand of Ragnaros")) {
                    items[i].quality = items[i].quality - 1;
                }
            }
        }
    }
}
```



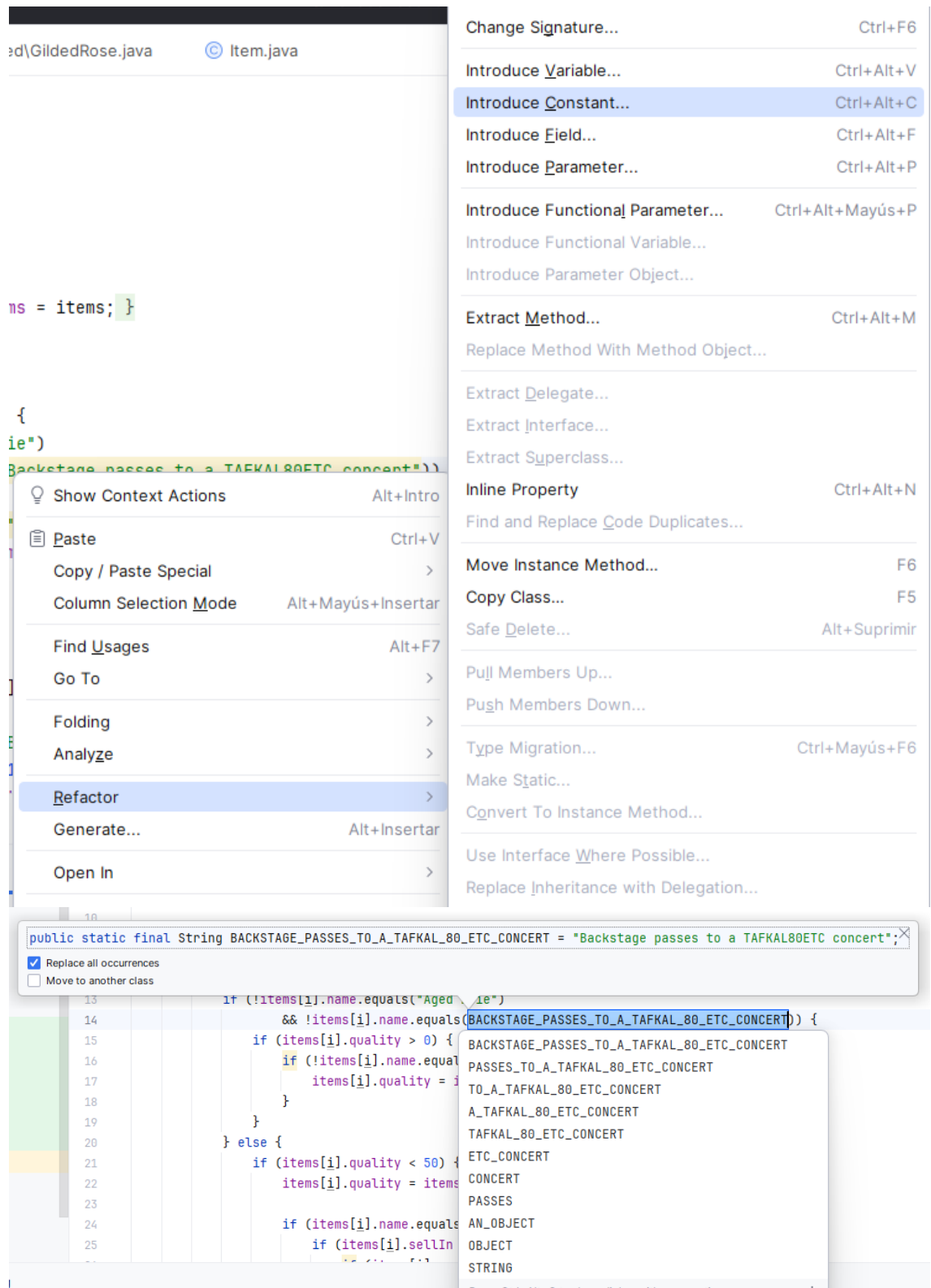
2. Antes de hacer ninguna modificación, comprueba que el código pasa satisfactoriamente todas las pruebas situadas en la carpeta *test*. Las pruebas de la carpeta *original* se ejecutarán sobre la carpeta *original* de *main*, mientras que las pruebas de la carpeta *refactored* se ejecutarán sobre la carpeta *refactored* de *main*, que será donde realicemos las modificaciones.
3. **Ahora nos centramos en las sugerencias relacionadas con los strings, las constantes y las variables.** Realizaremos las modificaciones al código con las instrucciones del profesor. Una vez realizadas las sugerencias, habrán aparecido nuevas constantes para los strings, como la que se muestra a continuación:

```
class GildedRose {
    public static final String BACKSTAGE_PASSES = "Backstage passes to a TAFKAL80ETC concert";
    Item[] items;

    public GildedRose(Item[] items) {
        this.items = items;
    }

    public void updateQuality() {
        for (int i = 0; i < items.length; i++) {
            if (!items[i].name.equals("Aged Brie")
                && !items[i].name.equals(BACKSTAGE_PASSES)) {
                if (items[i].quality > 0) {
                    items[i].quality = items[i].quality - 1;
                }
            }
        }
    }
}
```

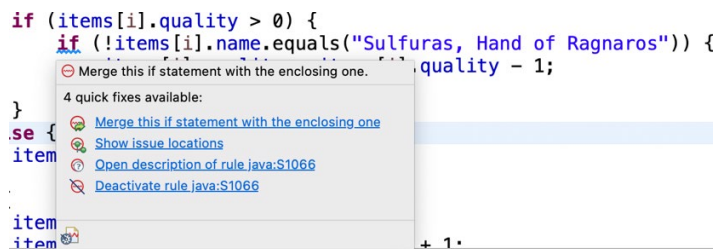
Para introducir la constante, podemos usar la funcionalidad de Refactorización del propio IDE:



4. Haz lo mismo con las demás constantes. ¿Hay algo que no detecte SonarLint y sí debería detectarlo?
5. Tras cada cambio o sugerencia de SonarLint aceptada, recuerda ejecutar las pruebas para comprobar que el comportamiento del código no ha cambiado. Esto es especialmente importante con los cambios realizados a mano.

Ejercicio 5: Reducir la complejidad cognitiva

1. **Identificar el problema.** Si ponemos el ratón sobre la función `updateQuality`, veremos un mensaje SonarLint que dice “Refactor this method to reduce its Cognitive Complexity from 69 to the 15 allowed.”
2. Como se explicó en clase de teoría, la complejidad cognitiva incrementa con los bucles y las condiciones y el anidamiento de estos. Por tanto, tendremos que simplificar las condiciones tanto como sea posible de las siguientes maneras:
 - 2.1. **Condensar condiciones anidadas.** Los anidamientos de sentencias `if`, `for`, `switch` y `try` son la razón de lo que se conoce como “código espagueti”. Este código es difícil de leer y refactorizar y, por tanto, de mantener. Vemos ejemplos con SonarLint en el código y podemos aplicar sus sugerencias:



Tras lo cual, el código resultante es:

```
if (items[i].quality > 0 && (!items[i].name.equals("Sulfuras, Hand of Ragnaros"))) {  
    items[i].quality = items[i].quality - 1;  
}
```

Pero debemos tener cuidado, ya que abusar de esta estrategia puede hacer el código menos legible.

- 2.2. SonarLint no detecta todas las sugerencias posibles, así es que habrá que aplicar manualmente ciertas refactorizaciones.
- 2.3. **Definir métodos o variables que implementen condiciones complejas.** Cuando la legibilidad del código se ve dificultada por condiciones complejas, podemos encapsular dichas condiciones en nuevos métodos que mejoren la legibilidad del código. Además, el aumentar la modularidad hace que el código se vuelva más mantenible, sea más fácil encontrar errores y se facilite la evolución del código. Esto hace que la complejidad cognitiva se reduzca.

En el código proporcionado, vemos que hay una condición que comprueba si la calidad del item es mayor que 0 y su nombre no es “Sulfuras”:

```
if (item.quality > 0 && (!item.name.equals(SULFURAS))) {
```

Con esto, podemos definir un método llamado `qualityAboveZeroNoNameSulfuras`:

```

/**
 * Checks whether the item's quality is >0 and it's name is not Sulfuras
 * @param item
 * @return true when the item's quality is >0 and it's name is not Sulfuras
 */
1 usage
public boolean qualityAboveZeroNoNameSulfuras (Item item){
    return item.quality > 0 && !item.name.equals(SULFURAS);
}

```

Hay que tener precaución, ya que siempre se podrían introducir problemas si no se hace con cuidado. Por ello, siempre debemos probar el código tras modificaciones de este tipo.

- 2.4. **Añadir comentarios.** Añade comentarios JavaDoc escribiendo `/**` y pulsando `<intro>` en la línea anterior al método. Añade comentarios normales (con `//`) para explicar todo lo que has hecho hasta ahora y continúa comentando tu código. Asegúrate de añadir suficientes comentarios para que el código se entienda, pero no más de los necesarios
- 2.5. **Continúa mejorando el código** hasta que la complejidad cognitiva de la función `updateQuality` se reduzca a 30 o menos.
- 2.6. Recuerda comprobar que el comportamiento del código no se ha modificado ejecutando las **pruebas** `GildedRoseTest` después de cada cambio.

Evaluación

- Entrega el proyecto comprimido que contiene el código de la clase `GildedRose.java` con las modificaciones realizadas y con toda la documentación.

Recursos adicionales

- Website oficial de SonarLint: <https://www.sonarsource.com/products/sonarlint/>
- What is Clean Code? <https://www.sonarsource.com/solutions/clean-code/>
- Catálogo de refactorizaciones: <https://refactoring.guru/es/refactoring>